

Catálogo de Átomos de Confusão em SystemVerilog

1.

```
module mux (  
    input  logic [1:0] selector,  
    input  logic      enable,  
    output logic [7:0] out  
);  
  
    logic [7:0] decoded;  
  
    always_comb begin  
        case (selector)  
            0: decoded = 8'd10;  
            01: decoded = 8'd20;  
            10: decoded = 8'd30;  
            2: decoded = 8'd40;  
            default: decoded = 8'd100;  
        endcase  
  
        if (enable)  
            out = decoded;  
        else  
            out = 8'd0;  
        end  
    end  
endmodule
```



```
module mux (  
    input  logic [1:0] selector,  
    input  logic      enable,  
    output logic [7:0] out  
);  
  
    logic [7:0] decoded;  
  
    always_comb begin  
        case (selector)  
            2'b00: decoded = 8'd10;  
            2'b01: decoded = 8'd20;  
            2'b10: decoded = 8'd30;  
            2'b11: decoded = 8'd40;  
            default: decoded = 8'd100;  
        endcase  
  
        if (enable)  
            out = decoded;  
        else  
            out = 8'd0;  
        end  
    end  
endmodule
```

Catálogo de Átomos de Confusão em SystemVerilog

2.

```
module top (  
    output logic [31:0] out0,  
    output logic [31:0] out1  
);  
  
    logic [1:0][31:0] A;  
  
    initial begin  
        A = '{1'b1, 1'b1};  
  
        out0 = A[0];  
        out1 = A[1];  
    end  
  
endmodule
```



```
module top (  
    output logic [31:0] out0,  
    output logic [31:0] out1  
);  
  
    logic [1:0][31:0] A;  
  
    initial begin  
        A[0] = 1'b1;  
        A[1] = 1'b1;  
  
        out0 = A[0];  
        out1 = A[1];  
    end  
  
endmodule
```

Catálogo de Átomos de Confusão em SystemVerilog

3.

```
module example;

function int maxx(int a, int b);
    int max = a;

    if (b > max)
        max = b;

    return max;
endfunction

initial begin
    int r1, r2, r3;

    r1 = maxx(3, 7);
    r2 = maxx(1, 2);
    r3 = maxx(0, 0);
end

endmodule
```



```
module example;

function automatic int maxx(int a, int b);
    int max = a;

    if (b > max)
        max = b;

    return max;
endfunction

initial begin
    int r1, r2, r3;

    r1 = maxx(3, 7);
    r2 = maxx(1, 2);
    r3 = maxx(0, 0);
end

endmodule
```

Catálogo de Átomos de Confusão em SystemVerilog

4.

```
module example (  
    input logic enable,  
    output int out  
);  
    int result;  
    byte in = -5;  
  
    always_comb begin  
        result = in + 1'b1;  
  
        if (enable)  
            out = result;  
        else  
            out = in;  
    end  
endmodule
```



```
module example (  
    input logic enable,  
    output int out  
);  
    int result;  
    byte in = -5;  
  
    always_comb begin  
        result = in + 1;  
  
        if (enable)  
            out = result;  
        else  
            out = in;  
    end  
endmodule
```

Catálogo de Átomos de Confusão em SystemVerilog

5.

```
module example;

    int lo, med, hi;
    bit result;
    int selected;

    initial begin
        lo = 20;
        med = 224;
        hi = 164;

        result = (lo < med < hi);

        if (result)
            selected = med;
        else
            selected = hi;
    end
endmodule
```



```
module example;

    int lo, med, hi;
    bit result;
    int selected;

    initial begin
        lo = 20;
        med = 164;
        hi = 224;

        result = (lo < med < hi);

        if (result)
            selected = med;
        else
            selected = hi;
    end
endmodule
```

Catálogo de Átomos de Confusão em SystemVerilog

6.

```
module example (  
    input logic [3:0] instruction,  
    output logic [2:0] opcode  
);  
  
    always_comb begin  
        casex (instruction)  
  
            4'b0???:  
                opcode = 3'b001;  
  
            4'b1000:  
                opcode = 3'b010;  
  
            default:  
                opcode = 3'b111;  
  
        endcase  
    end  
endmodule
```



```
module example (  
    input logic [3:0] instruction,  
    output logic [2:0] opcode  
);  
  
    always_comb begin  
        case (instruction) inside  
  
            4'b0???:  
                opcode = 3'b001;  
  
            4'b1000:  
                opcode = 3'b010;  
  
            default:  
                opcode = 3'b111;  
  
        endcase  
    end  
endmodule
```